极速焕彩

从题面拆解到可移植 SIMD 的完整探索报告

2026 Hackathon 决赛 · 第 16 题

三名本科生团队 · 18 天备赛 · 24 小时正式赛程

基于 43 份独立运行记录、24 个当前候选、696 条逐图统计

报告数据基准 Git：f73b8dd81a360521098ee985d68bcb3b548ccb19

工作目录：~/problem_probe/16 | 报告源文件：report/exploration.typ

目录

1 执行摘要、题面与证据边界 3
1.1 题面约束与我们如何解释 3
1.2 白盒数值问题与主要方法 3
1.3 证据分层 3
2 工程架构、数据和回溯系统 3
2.1 从原始 LUT 到最终图像 3
2.2 数据集、真值和文件完整性 4
2.3 测试边界与曾经发现的问题 4
2.4 Pixi、容器与独立日志 4
3 探索过程：从朴素采样到当前严格门禁 4
3.1 第一阶段：题面固化和可运行基线（10 月 12 日） 4
3.2 经典、现代与前沿路线的收获 4
3.3 场景迁移的反例 5
3.4 第二阶段：运行时与跨语言探索 5
3.5 第三阶段：审计和五项新发展（10 月 13 日） 5
4 当前结果、性能解释与选题决策 6
4.1 24 候选的统一精度与速度协议 6
4.2 准确率与最坏颜色的权衡 6
4.3 单级复合与历史非统一测量 6
4.4 Zig 宽度、分块与调度 7
4.5 三人团队的锁题与比赛策略 7
5 团队使用指南与复现顺序 7
5.1 开箱、构建和测试 7
5.2 跑候选、精度与吞吐量 8
5.3 查日志、回退和团队同步 8
5.4 从官方赛题包过渡 8
5.5 重建本报告 8
6 原始数据附录与审计线索 9
6.1 附录 A：当前 24 候选完整表 9
6.2 附录 B：历史六场景的 10 图代理对照 9
6.3 附录 C：可移植 Zig 的完整 36 组扫描 9
6.4 附录 D：原始归档与 SHA-256 10
6.5 附录 E：Git 探索时间线 10
6.6 附录 F：全部独立运行台账 11

1 执行摘要、题面与证据边界

本报告记录第 16 题“极速焕彩”的完整工程探索：从原始 HTML 快照提取题面，建立模拟四级 RGB 查找表和图像集，比较朴素采样、传统基函数、精确四面体最小二乘、整数精化、残差补偿及单级复合，再将运行时从 C++ 标量推进到可移植 Zig `@Vector`。结果来自固定代码提交和独立实验日志；失败路线、历史测试口径和修复记录均保留。

当前选题判断是“有条件可选”。三人团队已具备可运行的四阶段主线和自动精度/吞吐量筛选器，适合在 18 天准备期内完成接口对接与彩排。正式阈值、LUT 文件格式、每图色差判定粒度、计时边界及单级复合是否合法仍未公布。本文所有“通过”只表示在自建 `isp-0`、29 张图和**每图平均 RGB 距离严格小于 2.0** 的本地示例门槛下通过，不能替代官方验收。

1.1 题面约束与我们如何解释

`PROBLEM.md` 从用户给的 `source.htm` 提取题目与公式。输入是一张 1024×768 RGB 图、四个 256^3 整型色彩空间；每个空间须稀疏化为 `m×m×m`，其中 17≤m≤33；计算经过四次颜色映射，逐步输出仍为整数。插值由参赛队设计。题目采用 RGB 通道欧氏距离：

$$\Delta E = \sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}$$

题面写“ΔE < Threshold”，但没有明确比较对象是每个像素、每张图的平均、分位数还是最大值。它还允许“等价计算优化次数”，同时要求每步在稀疏空间完成；因此四阶段映射是当前默认合规候选，单级复合只作为需要裁判澄清的备选。原题给出 i7-14700、100 张图、1.5 s/张的参考性能条件；最快队伍才得速度满分，不能把“低于 1.5 s”写成满分保证。图像与 LUT 的实际编码、字节序、量化位深和 I/O 是否计时均待赛题包确认。

维度	已实现的本地代理与剩余规则问题
色差	逐像素 RGB 欧氏距离；门禁可选每图 mean、p99、max，并严格比较未舍入数值。官方判定口径未知。
参考变换	四个完整 256^3×3 `uint8` 表依次整数查表，预复合得到 `composite.lut256`；官方 LUT 表示未知。
稀疏空间	m∈{17,25,33}；节点 `uint8`、Q16 插值与序列化轴表。非均匀网格附加查找表是否允许未知。
性能	实际 C++/Zig 可执行程序处理 100 张图；整组重复轮次中位数用于筛选。官方 CPU 和 I/O 范围待核。
单级复合	能实测，但默认从自动获胜资格中排除；须确认“四次映射”等价优化的解释。

1.2 白盒数值问题与主要方法

对于固定网格和四面体划分，每个输入颜色只访问四个节点，插值前的输出是节点值的稀疏线性组合。项目用 Q16 权重拟合节点：`fit_ls_tetra` 在 64^3 RGB 子网格上构造稀疏正规方程，以共轭梯度求实数节点，再用 8 色节点分组的坐标下降优化取整后的目标。当前四阶段运行时每一步做 `uint8` 取整，所以整体级联并非单个凸优化问题；`fit_cascade_residual` 将前三阶段实际输出映射到完整四阶段真值，针对最终误差校正第四阶段。

朴素采样、盒滤波、三线性可分离最小二乘、Lagrange/B-spline 基、IRLS、误差驱动节点、Tucker/RBF/MLP/SIREN 分别测试了不同的逼近假设。我们没有把“小样本历史路线扫描”和“当前实际内核全色门禁”合并排名：前者用于理解场景差异，后者才是现阶段选择器依据。神经路线是有限预算下的探索，不代表对应论文或成熟产品的性能上限。

1.3 证据分层

- **题面事实**：`source.htm`、`PROBLEM.md` 的原文及整理备注；备注中的推断不是裁判结论。
- **历史探索**：`experiments/route_sweep.json` 的 189 条网格候选、45 条非网格候选、54 条历史吞吐量记录。其精度主扫描只用 10 张图和当时的 Python 评测，不能直接代替当前 29 张图门禁。
- **当前验证**：独立日志 52e23c0b 对 `isp-0` 的 24 个候选使用真实可执行输出、29 张图与全部 2^{24} RGB 颜色；达到本地示例门槛的合规候选才进入 100 图测速。
- **推断与未验证**：目标机器更快、单级复合可提交、正式图像分布接近模拟 ISP 等，均不能由本地数字推出。压力场景仍有明显高误差。

2 工程架构、数据和回溯系统

2.1 从原始 LUT 到最终图像

代码分为离线稀疏化和在线映射两层。`python/lutlab/data.py` 生成或核验输入图像与完整 LUT；`fit.py` 和 `cascade.py` 产出 `.slut`，其中包含节点、非均匀轴位置以及按原始浮点网格计算的 `idx/frac` Q16 表；`io.py` 和 `interp.py` 提供 Python

精确镜像。在线 `jshc` 的 C++17 层负责 PNG 读取、OpenMP 分块、四阶段流水及输出；默认插值内核是 Zig `@Vector(16, i32)`。C++26 `<simd>` 与 Rust `core::simd` 用于可移植性和速度对照。当前工作分支有 Rust Pixi 依赖，主分支没有；旧 AVX2/汇编尝试保存在归档提交/分支，当前主线不依赖架构特化代码。

`jshc cube` 对全部 16,777,216 种 RGB 颜色输出裸字节。`validation.py` 对比精确四阶段复合表，进一步在每张评测图上计算均值、RMS、p99、最大值和失败原因。`selection.py` 将拟合方法与实际插值内核成对评测：`ls_tri` 配 `tri_float`，四面体方法配 Zig。它只给通过用户指定阈值且规则许可的候选做吞吐量测试，按完整 100 张图的重复轮次中位数选方案。对于 Docker、macOS 和官方 CPU，本仓库还没有同等范围的实测结果。

2.2 数据集、真值和文件完整性

本地精度集是 24 张 Kodak 原图统一重采样后的图像，加 5 张覆盖色立方体、色相渐变、随机颜色、记忆色和中性灰边界的合成图，共 29 张。性能集从这些照片生成 100 张 1024×768 RGB 图。`isp-0/isp-1`、胶片、扭曲、裁剪和阶梯量化压力场景共六类模拟四阶段 LUT；`isp-0` 是本轮当前 24 候选全色验收的唯一场景。合成图和模拟 LUT 增强了覆盖，却不能代表未知的正式评测分布。

数据生成器默认要求 24 张 Kodak 原图全部获取；失败直接报错。只有显式 `--allow-synthetic-only` 才会生成降级数据。`data/manifest.json` 固定来源模式、种子和 155 个原图/图像/列表文件的 SHA-256；列表路径相对于项目根目录。`verify-data` 还检查 29/100 数量、尺寸与 RGB 模式、四个完整 LUT 的哈希，并重新复合验证 `composite.lut256` 的内容。约 1.8 GiB 图像和 LUT 本体由主仓库忽略；报告只存原始清单，不复制大型文件。

2.3 测试边界与曾经发现的问题

C++ 测试对 m=17/25/33、全部 2^{24} RGB 输入核对标量、C++26、Zig 4/8/16 lane 与 Rust 的位精确结果，并覆盖长度不满向量宽度的尾部及多线程级联。Python 测试补足 `.slut` 往返、序列化 Q16 表、损坏文件拒绝、缓存隔离和拟合行为。最终本地 `pixi run test` 的 CTest 1/1、Pytest 5/5 通过。

审计曾发现三个会污染结论的缺陷：Python 读取 `.slut` 后重算浮点轴表，少数 Q16 分数差 1；自适应与普通网格复用同一缓存目录；C++ 对损坏文件可能在检查前做越界指针运算。它们分别在 `866ca68` 与后续提交中修复，并把失败测试和修复复测保留在独立日志。基准输出还区分“各图跨轮取最短均值”和“完整 100 图最佳/中位轮次”，避免把前者当持续吞吐量。报告图 `docs/plots/cpp_throughput.png` 标识为历史数据，不用于当前默认 Zig 16 lane 排名。

2.4 Pixi、容器与独立日志

主 `pixi.toml` 锁定 Zig 0.17、Linux GCC 16.2 的 C++26、Rust 1.99、Python 3.12、CMake/Ninja 和科学计算包；ML 附加环境有 PyTorch。WSL/Linux 是已实测开发平台，macOS ARM 在清单中但尚无团队机器复验。Ubuntu 24.04 Dockerfile 作为备选，`.dockerignore` 排除数 GiB 的 Pixi、数据与日志；当前 WSL 没有 Docker，镜像未实测。`report/pixi.toml` 为 Typst 和 Noto CJK 字体创建独立环境，不改动算法锁文件。

`logs/` 有自己的 Git 仓库，主代码回退不会删除日志。本报告取数时共有 43 份 `run.json`，其中 1 份明确记录为失败；取数日志 Git 为 32f23355ab678e0696725ac871762a5be7a0ce57。每条记录包含命令参数、代码 SHA、脏工作区标记、CPU、运行时间、Pixi 锁哈希及原始输出/附属 JSON。`report/data/run_records_exact.tar.gz` 按原相对路径和原始字节归档本次所有日志文件；`manifests_exact.tar.gz` 固定输入清单，`experiments_exact.tar.gz` 固定历史实验 JSON。它们不包含被忽略的大型图像；第 6 章给出 SHA-256。

3 探索过程：从朴素采样到当前严格门禁

3.1 第一阶段：题面固化和可运行基线（10 月 12 日）

首个提交 `15937ee` 建立 Pixi、C++ 稀疏 LUT 读写、标量四面体/三线性核和内核测试；`ed0154b` 增加 Python 数据生成、拟合、级联、指标与独立日志。最简单的候选直接在 m^3 节点上从完整 LUT 取样，随后做运行时插值。它保证能跑通全链，是性能和误差的基准，但不是合理的最终拟合：取样坐标的非整数相位和四次取整叠加，可能使更大的网格反而更差。

`a51d0aa` 加入路线扫描与 Ubuntu 容器文件；随后多个提交把慢速的全色正规方程求解改为子网格、预加载图像并向量化 Python 评测，以便在备赛时间内完成多场景探索。每次改动后保留历史日志，而不是把较快的拟合耗时混进旧结果。`83464cb` 固化了 20 条路线、历史图表和 `ROUTES.md/EVALUATION.md`。这轮 `route_sweep.json` 有 189 条网格路线、45 条非网格路线；网格路线精度主扫描只覆盖每场景 10 张图，故本章以“找方向”解释，不当正式门禁。

3.2 经典、现代与前沿路线的收获

`isp-0` 的历史 10 图扫描中，m=17 朴素四面体 `R2` 的平均 ΔE 为 2.5907；可分离三线性最小二乘 `R5` 为 1.4558；与运行时四面体权重匹配、再做整数精化的 `R8` 为 1.4453；四级残差 `R11` 为 1.3901；单级复合 `R13` 为 1.1403。m=33 的 `R2` 仍为 2.3233，高于 m=25 的 1.6370，展示“节点更多并不自动减误”的具体反例。实际比较受网格相位、节点取整和四级传播共同影响。

Lagrange 三次张量基在 `isp-0` m=17 的历史 10 图平均 ΔE 为 1.5115；简化边界处理的二/三次 B-spline 分别为 4.9208、6.1873，提示实现与边界匹配的重要性，不能由此否定所有样条方案。Tucker r=10 在 m=17 为 1.5485，而 r=6 为 2.0152，节省参数伴随失真。三次/五次全局多项式分别为 9.6047/4.4887，216 中心 RBF 为 3.6923；短预算 3×32 MLP GELU 与 SIREN 为 22.8997/21.464。神经网络结果只反映当时的小样本、步数和配置，不是 DL 路线的收敛上限；在 24 小时比赛中继续无边界训练风险较高。

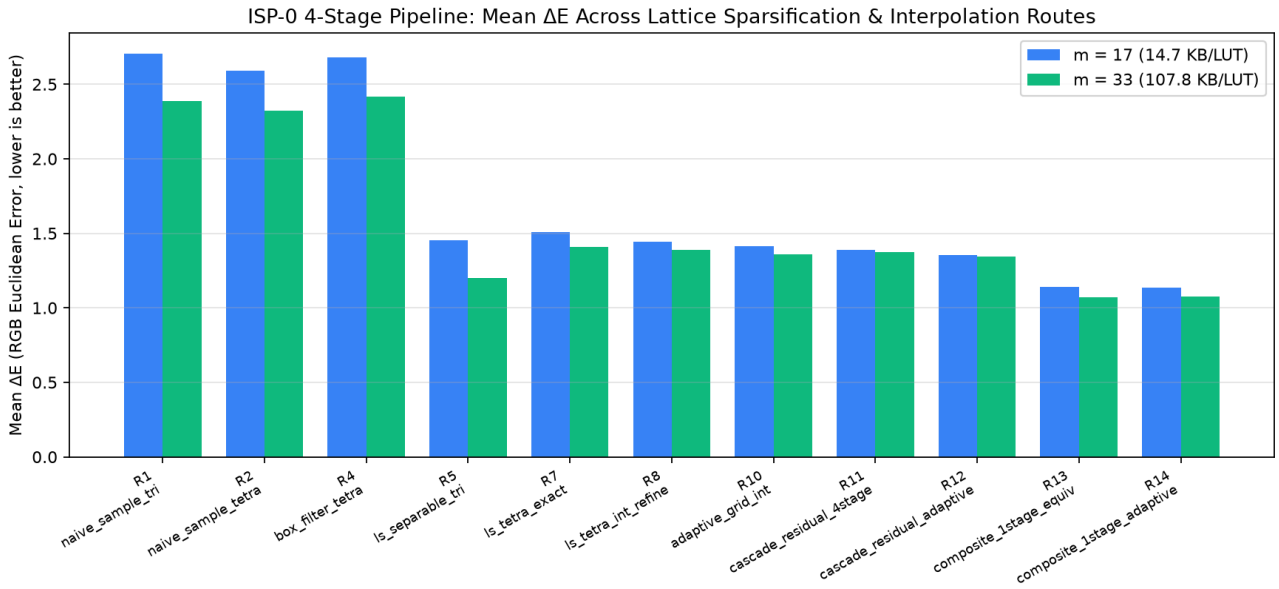


图 1　历史 `route_sweep.json` 的 `isp-0` 精度比较。图中是早期 Python 代理评测，不能与当前 29 图可执行程序门禁混成同一排名。

3.3 场景迁移的反例

历史数据中，`film-0` m=33 的复合 `R13` 平均 ΔE 为 0.8584，低于残差 `R11` 的 1.0906；但 `warp-0` m=17 的三线性 `R5` 为 1.6154，优于复合 `R13` 的 2.2658。`stress-0` m=17 的最好几条也约 7.6，远高于光滑 ISP 场景。这些数值是模拟场景下的历史 10 图均值，却已经足够否定“一个方法对所有 LUT 最优”的假设。团队应让官方数据进入同一个候选门禁，而不是预先固定 m 或插值核。第 6 章列出六场景 R2/R8/R11/R13 对照，原始 189 行保存在实验 JSON 归档。

3.4 第二阶段：运行时与跨语言探索

`82e8c69` 曾加入 x86-64 AVX2/手写汇编和原生 C++ 求解器；随后 `30f664b` 做了无架构特化的 C++17 版本，`969d046` 比较 C++26 `<simd>`、Zig `@Vector`、Rust `core::simd`。当前分支 `408a842` 用 Pixi 锁定 Zig、GCC 16/C++26 与仅分支内的 Rust，并把 Zig 设为默认。归档的 AVX2/汇编有历史价值，但当前方案遵守“语言设计/builtin/stdlib、不手搓 asm、不按架构特化”的用户约束。

早期多语言 `R14_multi_lang_portable_simd` 的汇总日志实际创建耗时只有 0.013 秒，缺少完整执行原始输出，不能据其极低耗时作当前排名。审计后在相同 m=33 四阶段、100 张图、三轮、20 线程的本机复测：Zig 整组最佳 143.983 ms，C++26 为 219.123 ms，Rust 为 246.026 ms（日志 `7bf37a1e`，当时默认 Zig 8 lane）。此比较说明当前机器的可移植 Zig 核有竞争力；它不能推出不同 CPU、编译器和官方环境下的固定胜序。

3.5 第三阶段：审计和五项新发展（10 月 13 日）

前次审计修复 Q16 序列化表被 Python 重算、模型缓存碰撞、损坏 `.slut` 读取和历史基准口径错误。随后按用户要求完成五项发展：`727df3e` 增加真实内核全色精度门禁、24 候选筛选、尾部重加权与网格误差布点；`8247112` 固定数据哈希、一键工作流和 Zig 宽度/块/调度扫描；`a949f22` 让末级残差针对最终输出加权；`ca638ea` 把速度排序从最佳单轮改为整组中位数；`51baedf` 在全色位精确校验后把默认 Zig 切到 16 lane。后续提交补正文档、精确复合校验和相对路径运行。

新方法没有被包装为全胜：m=17 的逐级高误差加权使最差单图均值从普通 `tetra` 的 1.702 升至 2.035，越过本地 2.0 门槛；末级残差加权把全色最大误差从 `residual` 的 10.25 略降到 10.10，却让最差图均值从 1.547 升到 1.600；m=33 误差驱动网格则将全色最大误差从普通 `tetra` 的 9.70 降到 7.35。改进方向取决于官方门槛究竟看均值、p99 还是最大值。所有这些数字取自同一次 24 候选扫描，而非混用历史 Python 代理。

4 当前结果、性能解释与选题决策

4.1 24 候选的统一精度与速度协议

本轮固定 `isp-0`、29 张 1024×768 RGB 精度图、100 张同尺寸吞吐图、20 线程。每个候选先用实际 `jshc cube` 对全部 2^{24} 颜色生成输出，并以四个完整 LUT 的精确复合为真值；将每图**未舍入的平均 ΔE < 2.0** 作为本地示例硬门槛。过线且合规的候选再做 3 次完整 100 图映射，选取整组时间中位数。PNG 解码单独计时，整组映射时间包含每张工作缓冲拷贝，但不含 PNG 解码/写盘或离线拟合。第 6 章展示全部 24 行、696 条逐图结果的 CSV 及原始 JSON 归档。

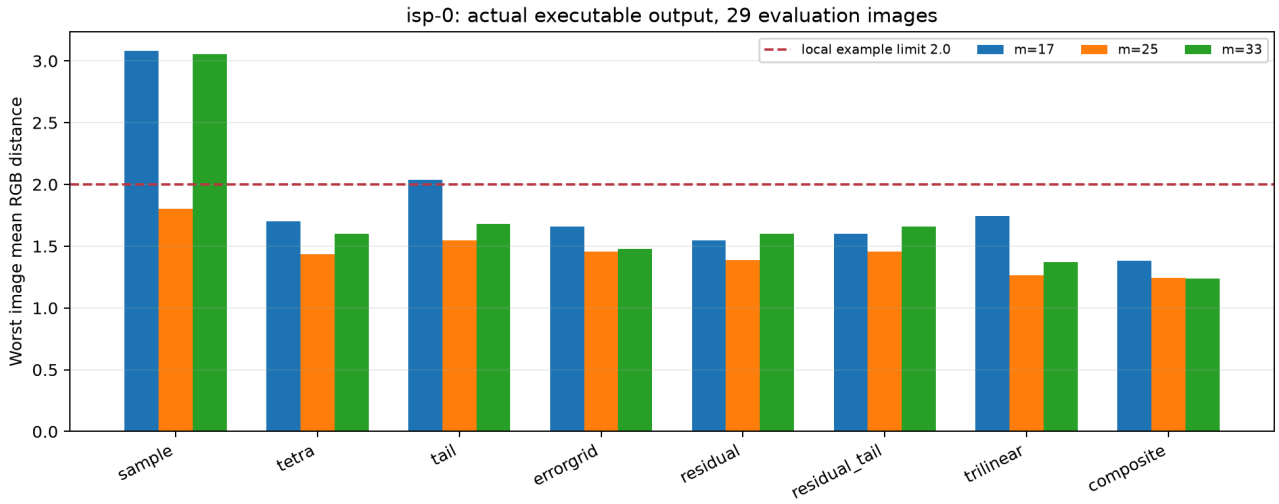


图 2　当前 24 候选的最差单图平均 ΔE；红线是本地示例阈值 2.0。`composite` 达到数值门槛但默认不视为规则许可。

m	路线	门禁	最差图均值	全色最大	100 图中位 ms
17	`sample`	失败	3.079	10.49	—
17	`tetra`	通过	1.702	15.26	129.624
17	`errorgrid`	通过	1.661	15.03	128.187
17	`residual`	通过	1.547	10.25	**126.332**
17	`residual_tail`	通过	1.600	10.10	131.648
25	`residual`	通过	1.387	9.06	130.459
33	`errorgrid`	通过	1.478	**7.35**	132.069
25	`trilinear`	通过	1.263	9.27	745.164
17	`composite`	规则待确认	1.384	15.13	未排序

本轮最快的合规候选是 m=17 四阶段 `residual`，整组中位 126.332 ms，换算 1.263 ms/图。这是**当前机器上的本地选择结果**，不宜据 2–5 ms 的候选间差值宣称稳健胜出：较早的 5 次重复扫描由 `residual_tail` 获胜，但它使用当时的 Zig 8 lane 默认内核，不能与本轮直接比较。本轮 m=33 普通 `tetra` 为 271.531 ms，远高于同规模 `errorgrid` 的 132.069 ms，值得在同一内核下单独复测。计时受 WSL 调度、核心混合与温度干扰；需要在目标机器上随机化顺序、重复整个候选测试，并重新记录中位数和尾部。`selection.json` 留存所有候选的原始 bench 对象，表格只为阅读而舍入。

4.2 准确率与最坏颜色的权衡

普通 `tetra` m=17 的最差图均值为 1.702，全色最大 15.26；末级残差补偿将二者变为 1.547 和 10.25，说明四次取整误差可以针对最终输出吸收。逐级尾部重加权 `tail` m=17 的最差图均值反而 2.035，失败；它在 m=25/33 通过，但并非原始 `tetra` 的全面改良。m=33 `errorgrid` 的全色最大 7.35，比普通 m=33 `tetra` 的 9.70 小，且全色 p99 从 3.606 降到 3.317；若官方按最大值或尾部色差限制，它可能比本地均值最快的 m=17 更值得选择。

精度来自真实可执行输出，不是只用 Python 插值推测。初次 m=33 基线门禁在颜色 `(127,254,97)` 测得全色最大 9.695；全部候选的前 20 个最差 RGB 坐标在 `report/data/cube_hotspots.csv`。它们可用于赛中定位官方 LUT 的局部折线或截断区，但不应仅对当前模拟颜色做定点“特判”。

4.3 单级复合与历史非统一测量

将四个完整 LUT 离线精确复合后再拟合一个 m^3 表，可以显著减少在线查表次数。当前 24 候选中 m=17 `composite` 的最差单图均值 1.384、全色均值 1.120；m=25/33 的全色 p99 均为 2.449。它们符合**本地数值门槛**，但默认状态为 `ineligible_composite`，没有参与本轮性能排名。较早在 `--allow-composite` 下的 Zig 8 lane 单独试验给出 m=17 整组

最佳 43.515 ms（日志 `0d6ba5f8`）；那是不同提交和最佳单轮口径，不能与上表的 Zig 16 lane 三轮中位数直接计算加速比。只有官方明确允许等价单次映射后，才能把它放进提交候选。

4.4 Zig 宽度、分块与调度

当前内核仅用 Zig `@Vector`、内建选择/最值与语言级泛型。36 组配置在 m=17 四阶段上扫描 lane 4/8/16、块 1024/4096/8192、OpenMP static/dynamic,1、4/20 线程；每组处理 100 图×3 轮。4 线程下各宽度的最快完整轮次分别约为 339.947、265.080、236.600 ms；20 线程下最快值为 125.877、129.343、123.056 ms，抖动明显。固定 4096 块与动态调度后，做 7 次整组复测：

线程	8 lane 中位 ms	16 lane 中位 ms	16 lane 改善
4	257.972	243.348	5.7%
20	141.227	135.011	4.4%

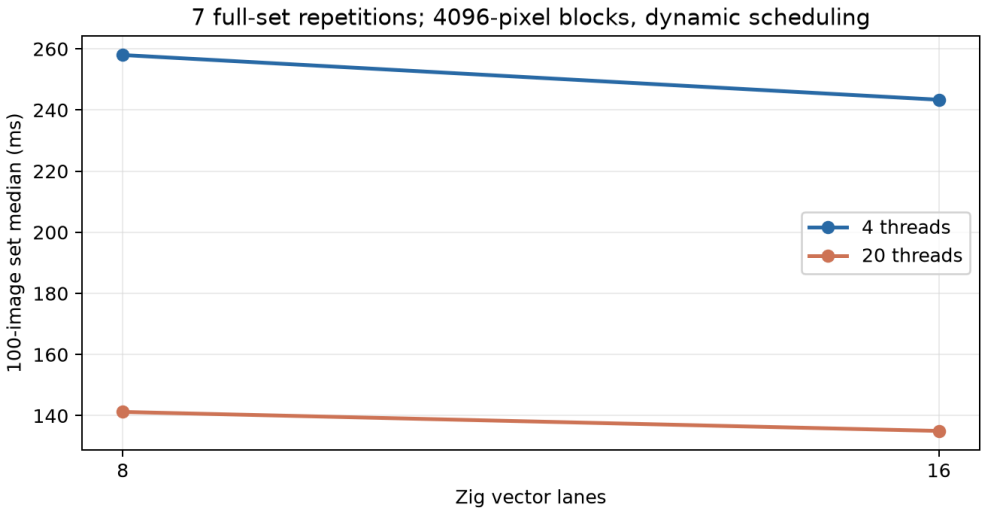


图 3　固定 4096 像素块、动态调度，7 轮完整 100 图中位时间。该图是当时 Zig 8/16 显式路径对照；后来把 16 lane 设为 `tetra_zig_simd` 默认。

C++ 全色测试已在 m=17/25/33 上确认 4/8/16 lane 与标量逐字节一致。因而 `51baedf` 将 16 lane 设为默认，保留显式 `tetra_zig_simd8`，让不同 CPU 可复验。更宽向量的收益并非架构无关的定律；在官方机器上应再次测 lane、线程与调度。

4.5 三人团队的锁题与比赛策略

48 小时内可考虑选择，但必须获得关键规则答复。 已有可运行四阶段实现、严格门禁、模型缓存、数据哈希和回退版本；主要未知不是“能否写出算法”，而是官方文件接口与色差规则。立即向主办方确认四项：输入 LUT 的通道位深及索引顺序、Threshold 对每图的统计口径、能否使用非均匀轴辅助表、能否离线复合四级。若规则与当前代理不同，48 小时内要重评可行性。

准备期三人分工：A 负责拟合和准确率，维护整数精化/残差/网格搜索；B 负责 C++/Zig 对接、官方机器性能和可移植构建；C 负责数据、黄金真值、独立日志和提交验收。第 1–3 天冻结官方格式适配和原始数据哈希；第 4–10 天按官方色差统计选择 m 与拟合；第 11–15 天在目标机器调 lane、分块和线程；最后 3 天做全量彩排与回退。正式 24 小时先通过官方门禁，再只对合格候选测速；最后数小时停止扩展算法，核对二进制、日志、输入输出与文档。

若官方测试含强量化阶梯、与本地 `stress-0` 相似，则当前方法在历史 10 图代理中的平均 ΔE 仍约 7.4–7.8，不能以 `isp-0` 结果声称稳过。若官方阈值比 2.0 严格，当前 24 候选可能全不合格。上述两种情况应优先解决精度和合法性，而非继续争取几十毫秒吞吐量。

5 团队使用指南与复现顺序

5.1 开箱、构建和测试

进入 `~/problem_probe/16`，先安装 Pixi，使用项目已锁定的主环境。Windows 队员以 Arch/Ubuntu WSL2 执行主线；原生 macOS ARM 在清单中，但必须由持有该平台的队员自行实测 C++26/Zig/Rust 构建。`pixi run test` 编译当前默认 Zig 16 lane、C++26 和 Rust 对照核，并执行 C++ 全色及 Python 测试。不要将不同机器的性能时间并排宣称速度提升。

```
cd ~/problem_probe/16
pixi install -a
pixi run test
pixi run verify-data
```

首次没有数据时，`competition-check` 会先获取完整 Kodak 图像、构造 29/100 张图和指定的模拟场景，再测试、核验和筛选；下载失败不会静默退回合成数据。首次构造可能需较长时间及约 1.8 GiB 磁盘空间。已有旧数据且缺清单时，可在确认来源后执行 `pixi run adopt-data`；该操作只为现有文件建哈希，不提供上游来源证明。

5.2 跑候选、精度与吞吐量

官方 Threshold 未公布，下面的 `2.0` 仅用于复现实验，不能原样作为比赛门槛。官方明确统计粒度后用 `--metric mean|p99|max` 指定；如裁判允许单级复合，再显式加 `--allow-composite`。

```
pixi run competition-check --scenario isp-0 --threshold 2.0 --metric mean --threads 20
pixi run select --scenario isp-0 --m 17 25 33 --profiles tetra errorgrid residual residual_tail --
threshold 2.0 --threads 20 --reps 5
pixi run validate --scenario isp-0 --threshold 2.0 --threads 20 data/models/isp-0/
residual_ls_tetra_int_m17/stage{1,2,3,4}.slut
pixi run zig-perf --widths 4 8 16 --blocks 1024 4096 8192 --schedules static dynamic,1 --threads 4 20 --
reps 3 data/models/isp-0/residual_ls_tetra_int_m17/stage{1,2,3,4}.slut
```

`select` 为缺少缓存的候选自动调用拟合，再做真实内核验证。`validate` 留下每图数据、全色统计、最差颜色和模型 SHA；失败时进程返回非零。`zig-perf` 是长时间测量，应在机器负载稳定时独立运行，不能和其他 CPU 工作并行。`jshc bench` 的 `set_ms_p50_rep` 是完整 100 图轮次中位数；`apply_ms_mean` 是每张图跨轮最佳时间的平均，数值更乐观，不能用它排序提交方案。

单个方法可用 `pixi run fit --scenario isp-0 --mode residual --m 17 --method ls_tetra_int` 生成；`--grid error` 选择误差驱动非均匀网格。复现历史基线时，当前代码重新拟合的 m=17/25/33 共 12 个 `.slut` 与存档 `bench_models` 逐字节相同，日志 `d4e6375c`、`5ce1f5e1`、`62445746`。官方数据到手后须重新拟合，不要提交当前模拟表。

5.3 查日志、回退和团队同步

```
pixi run logs
python -m lutlab.runlog show 52e23c0b
```

`logs/runs/<UTC>-<任务>-<ID>/` 下 `run.json`、`selection.json`、`validation.json`、`sweep.json` 和 `stdout.log` 保存完整口径；`competition-check` 另有 `test.log`。`logs/` 的 Git 历史独立于主代码仓库，主代码 `git reset` 不会撤销一次实验，也不会把旧日志自动推送给队友。团队应单独备份或共享日志仓库与生成数据清单；对比结果时核对模型哈希、数据哈希、代码提交、线程数、重复次数和阈值。新方法若失败，回到有完整哈希与门禁的上一候选，而非凭文件名 “stage1.slut” 猜版本。

5.4 从官方赛题包过渡

先登记官方四个 LUT 的字节尺寸、位深、索引顺序及 SHA，再用少量人工颜色构造黄金查表样例，确认参考实现与主办方样例一致。按官方文件格式做适配时，先保持 `io.py`/C++ 的一致性和损坏文件拒绝；随后用公布的 Threshold 和判定粒度替换本地参数，重新跑全色或主办方指定集，再测 100 图。若官方把拟合、PNG I/O 或模型初始化计入性能，必须另建同边界计时，当前 `set_ms_p50_rep` 不足以描述端到端提交时间。

官方数据和接口未收到之前，不应对 `PROBLEM.md` 的未明确部分作静默假设。最少需要四阶段模式过线；单阶段复合只在书面规则明确后参与选型。保留 `stress-0`、`warp-0` 测试用于发现失败模式，但不能以它们代替官方盲测。

5.5 重建本报告

本报告用独立 `report/pixi.toml` 安装 Typst 与 Noto CJK 字体。生成器在**已提交代码和独立日志固定**后读取 `logs/`，写入 CSV、固定时间戳的原始文件归档、SHA 索引和图；它不修改实验日志。此版本固定主扫描运行 ID 52e23c0b；重新做新一轮官方扫描时，应明确更新 `scripts/build_typst_data.py` 的固定 ID，不能偷偷把旧表换成新日志。

```
pixi run python scripts/build_typst_data.py
pixi run --manifest-path report/pixi.toml compile
pdfinfo report/exploration.pdf
```

若只想阅读，`report/exploration.pdf` 已是编译好的 A4 文档；`report/data/DATA_INDEX.md` 列出归档与 CSV 的 SHA-256。`report/data/run_records_exact.tar.gz` 可离线取回本报告取数时的运行记录。图像本体和 LUT 本体不在报告包内，仍应依照 `data/manifest.json` 单独保存或重新构造并核验。

6 原始数据附录与审计线索

正文引用的当前扫描固定为运行 52e23c0b，源提交 f73b8dd81a360521098ee985d68bcb3b548ccb19，独立日志提交 32f23355ab678e0696725ac871762a5be7a0ce57。这两个提交的用途不同：前者确定生成报告时的代码，后者确定已冻结的运行记录。共有 43 次运行，失败 1 次；其失败状态也收录，不将其改写成成功。`report/data/` 的归档保留原字节和相对路径，CSV 只是便于检索的派生视图。当前 24 候选产生 696 条逐图行、480 条最坏颜色行。表格显示小数经过舍入；做阈值复核应使用原始 `selection.json`。

6.1 附录 A：当前 24 候选完整表

门禁为 `isp-0`、29 张图、每图平均 ΔE 严格小于本地示例值 2.0。`ineligible_composite` 表示数值精度合格但规则未确认，速度栏缺失；`fail_accuracy` 的速度栏同样缺失。完整浮点精度、29 张逐图结果、模型哈希和全色最坏 RGB 值分别在原始 `selection.json`、`image_results.csv` 与 `cube_hotspots.csv`。

m	路线	本地门禁	阶段模式	最差图均值	全色均值	全色 p99	全色最大	100 图中位 ms	100 图最佳 ms
17	sample	fail_accuracy	seq	3.079	2.587	5.477	10.488	—	—
17	tetra	pass	seq	1.702	1.460	4.123	15.264	129.624	126.368
17	tail	fail_accuracy	seq	2.035	1.542	5.099	15.427	—	—
17	errorgrid	pass	seq	1.661	1.351	4.123	15.033	128.187	125.220
17	residual	pass	residual	1.547	1.383	3.742	10.247	126.332	126.311
17	residual_tail	pass	residual	1.600	1.419	3.742	10.100	131.648	119.844
17	trilinear	pass	seq	1.742	1.487	4.472	13.304	743.134	693.729
17	composite	ineligible_composite	composite	1.384	1.120	4.000	15.133	—	—
25	sample	pass	seq	1.805	1.608	4.123	8.124	142.890	136.137
25	tetra	pass	seq	1.435	1.247	3.606	11.225	136.788	132.362
25	tail	pass	seq	1.547	1.300	3.742	12.166	161.050	140.468
25	errorgrid	pass	seq	1.458	1.257	3.317	11.358	139.085	129.968
25	residual	pass	residual	1.387	1.224	3.162	9.055	130.459	129.608
25	residual_tail	pass	residual	1.456	1.265	3.317	10.050	189.725	138.873
25	trilinear	pass	seq	1.263	1.116	3.317	9.274	745.164	720.178
25	composite	ineligible_composite	composite	1.244	1.029	2.449	11.091	—	—
33	sample	fail_accuracy	seq	3.056	2.159	5.000	9.165	—	—
33	tetra	pass	seq	1.601	1.323	3.606	9.695	271.531	254.012
33	tail	pass	seq	1.682	1.339	3.606	9.695	145.402	139.554
33	errorgrid	pass	seq	1.478	1.255	3.317	7.348	132.069	128.620
33	residual	pass	residual	1.599	1.303	3.317	10.488	144.119	136.939
33	residual_tail	pass	residual	1.659	1.343	3.317	11.446	132.772	131.465
33	trilinear	pass	seq	1.370	1.182	3.317	9.220	723.273	721.001
33	composite	ineligible_composite	composite	1.239	1.014	2.449	9.055	—	—

6.2 附录 B：历史六场景的 10 图代理对照

`route_sweep.json` 按场景与 m 对比朴素四面体、整数最小二乘、四级残差和单级复合。它使用当时的 Python 代理、每场景 10 张图，不与附录 A 的当前门禁数值合并排名。单级复合仍有规则风险；`stress-0` 的误差高是当前算法族的重要能力边界。

场景	m	朴素 R2	整数 LS R8	残差 R11	复合 R13
clip-0	17	3.531	1.946	1.908	1.607
clip-0	33	2.805	1.675	1.718	1.355
film-0	17	2.110	1.160	1.111	1.000
film-0	33	1.727	1.136	1.091	0.858
isp-0	17	2.591	1.445	1.390	1.140
isp-0	33	2.323	1.387	1.376	1.073
isp-1	17	2.652	1.444	1.428	1.192
isp-1	33	2.165	1.276	1.337	1.013
stress-0	17	9.019	7.846	7.770	7.844
stress-0	33	8.161	7.767	7.478	7.486
warp-0	17	3.659	2.003	1.939	2.266
warp-0	33	2.649	1.409	1.436	1.238

6.3 附录 C：可移植 Zig 的完整 36 组扫描

本表来自 `37f9fba6`，每格是 100 张图、3 轮中的**最佳整组**毫秒，适合看候选范围而不宜做微小差异的最终排序。固定配置的 7 轮中位复测见正文图及 `generated/zig_repeat.csv`。默认内核后来改为 16 lane；这里宽度 8 和 16 使用当时的显式 ABI 符号。

线程	lane	块	调度	100 图最佳 ms
4	4	1024	static	339.947
4	4	1024	dynamic,1	341.719
4	4	4096	static	344.098
4	4	4096	dynamic,1	340.575

线程	lane	块	调度	100 图最佳 ms
4	4	8192	static	342.953
4	4	8192	dynamic,1	343.014
4	8	1024	static	270.820
4	8	1024	dynamic,1	265.306
4	8	4096	static	266.841
4	8	4096	dynamic,1	265.154
4	8	8192	static	268.466
4	8	8192	dynamic,1	265.080
4	16	1024	static	248.614
4	16	1024	dynamic,1	251.784
4	16	4096	static	241.881
4	16	4096	dynamic,1	240.063
4	16	8192	static	242.544
4	16	8192	dynamic,1	236.600
20	4	1024	static	136.809
20	4	1024	dynamic,1	129.791
20	4	4096	static	137.635
20	4	4096	dynamic,1	125.877
20	4	8192	static	136.116
20	4	8192	dynamic,1	206.378
20	8	1024	static	226.875
20	8	1024	dynamic,1	129.343
20	8	4096	static	130.547
20	8	4096	dynamic,1	139.088
20	8	8192	static	130.548
20	8	8192	dynamic,1	130.239
20	16	1024	static	138.348
20	16	1024	dynamic,1	123.056
20	16	4096	static	142.410
20	16	4096	dynamic,1	130.334
20	16	8192	static	142.458
20	16	8192	dynamic,1	137.298

6.4 附录 D：原始归档与 SHA-256

文件	SHA-256
`run_records_exact.tar.gz`	c2508b7102ab4b3bb00fb7bbea07f25ae551648f99fbe83abf32d25115d7af31
`manifests_exact.tar.gz`	cbdee2b59eb94eed019176cb5ab468eefa35aab01e050c7b1c4b64f63d9e2417
`experiments_exact.tar.gz`	df057ccd32cdaa03d02e4cb83d080eb0762c375f5c8a1904678901aabea519e2
`ledger.csv`	b59d7371f880db75780b5ffe6404823b31e8f968954112b1ad775737196884ba
`image_results.csv`	b11c2dd1f295264ae64d1b785ce6bcadf8d53a338f39e3ef12a1d46b89b9f636
`cube_hotspots.csv`	338714396d4adbcb4cdfc0168b987179c9fdcc010d9fbb2d2ee90e69651d8b88

历史 `experiments/route_sweep.json` 的独立 SHA-256 是 8b12d597fb0d1e83ff573b9e3d397ed45fcce71eb2e1ecc436a1fa2f696ac4a7。精确归档不包含被 Git 忽略的约 1.8 GiB 图像/LUT，也不等于官方比赛数据。文档级路径说明见 `data/DATA_INDEX.md`。

6.5 附录 E：Git 探索时间线

保留原始提交标题以查找实现、失败、修复和门禁的出现时间。提交日期与运行 UTC 时间不是同一口径；报告文稿本身在数据基准提交之后完成。

提交	日期	原始标题
15937ee	2026-10-12	Scaffold: pixi env, C++ sparse-LUT kernels (tetra scalar/AVX2, trilinear), CLI and kernel tests
ed0154b	2026-10-12	Add Python lutlab package (io, interp, luts, data, fit, cascade, metrics, runlog, cli) and parity tests
a51d0aa	2026-10-12	Add route sweep harness (routes.py) and Ubuntu 24.04 Dockerfile
25b6d0b	2026-10-12	Optimize sparse LS normal equation subgrid stride for 8x faster fitting
4a00ed1	2026-10-12	Speed up Python trilinear and tetrahedral evaluation in route sweep
a698172	2026-10-12	Preload images and use 64^3 subgrid in normal equations for fast full sweep
b525774	2026-10-12	Vectorize image batch evaluation and speed up adaptive_grid curvature profile
f5e415d	2026-10-12	Use int32 in apply_tetra_q16 and 2x2 subgrid for fast sweep evaluation
b135b61	2026-10-12	Vectorize RBF distance with BLAS gemm and bound PyTorch thread contention
83464cb	2026-10-12	Complete 20-route sweep, plots, ROUTES.md, and EVALUATION.md
82e8c69	2026-10-12	perf(r14): add x86-64 AVX2 inline assembly kernel tetra_asm and native C++17 fit-r14 solver
30f664b	2026-10-13	refactor(r14): implement architecture-agnostic, maintainable C++17 R14 kernel and modular fit_r14 solver

提交	日期	原始标题
969d046	2026-10-13	feat(simd): compare C++26 <simd>, Zig @Vector, and Rust core::simd portable 8-lane kernels
408a842	2026-10-13	build: lock Zig C++26 and branch-local Rust toolchain; default to Zig SIMD
96ef224	2026-10-13	docs: record Zig default toolchain experiment and validation
866ca68	2026-10-13	fix: preserve sparse LUT tables and separate adaptive models; audit evidence
b2bfe5f	2026-10-13	test: save modified parity input before C++ comparison
07b0b2a	2026-10-13	docs: clarify historical throughput chart with full-set timing
f1c75e8	2026-10-13	docs: publish readable historical throughput chart and audit results
727df3e	2026-10-13	feat: add strict validation selection tail fitting and portable Zig tuning knobs
8247112	2026-10-13	feat: make datasets verifiable and add one-command workflow and Zig sweep
a949f22	2026-10-13	feat: fit final-stage residuals with optional tail weighting
ca638ea	2026-10-13	perf: rank candidates by median whole-set time
51baedf	2026-10-13	perf: make parity-checked Zig 16-lane kernel default
5152fe8	2026-10-13	docs: report five development tracks and verify exact scenario composition
123a7df	2026-10-13	fix: resolve executable inputs from project root for portable lists
37dfdb0	2026-10-13	docs: record byte-identical baseline model regeneration
f494cdb	2026-10-13	docs: scaffold auditable Typst exploration report
a38f67d	2026-10-13	docs: finalize report wording and tracked evidence package
a9444be	2026-10-13	build: make Typst PDF compilation reproducible
f73b8dd	2026-10-13	build: normalize report CSV and mark PDF binary

6.6 附录 F：全部独立运行台账

按运行 UTC 开始时间列出全部 43 条。此表包含一次审计后测试失败以及后来修复成功的记录。完整 `params_json`、分支、脏状态和代码 SHA 在 `data/ledger.csv`；原始 stdout、差异和指标在 `run_records_exact.tar.gz`。

UTC 时间	ID	类型	状态	代码	秒
2026-10-12 14:32	f36df336	routes-sweep	OK	b135b614f479	1444.4
2026-10-12 15:46	41945a8f	r14-extreme-opt	OK	83464cb09799	49.4
2026-10-12 15:59	be07cfe8	r14-portable-clean	OK	82e8c69d257c	55.8
2026-10-12 16:23	5c01f571	R14_multi_lang_portable_simd	OK	30f664bce413	0.0
2026-10-12 17:58	58d73c29	zig-default-smoke	OK	408a84289172	0.1
2026-10-12 17:59	d7255729	pixi-test	OK	96ef224553b6	6.5
2026-10-12 18:04	7bf37a1e	audit-current-kernels	OK	96ef224553b6	32.8
2026-10-12 18:05	938fd46d	audit-sparse-roundtrip	OK	96ef224553b6	0.0
2026-10-12 18:08	3d7c9ef2	audit-postfix-test	FAIL	866ca68b0d59	10.9
2026-10-12 18:08	c5abdb6a	audit-postfix-test	OK	b2bfe5f4da44	6.5
2026-10-12 18:09	4f6227c7	audit-bench-format	OK	b2bfe5f4da44	0.1
2026-10-13 07:42	059a57e9	validate	OK	8247112df86e	2.6
2026-10-13 07:43	6adcd809	select	OK	8247112df86e	23.9
2026-10-13 07:43	79d8f430	fit	OK	8247112df86e	0.2
2026-10-13 07:43	b5cb8531	fit	OK	8247112df86e	0.1
2026-10-13 07:43	b5437413	fit	OK	8247112df86e	0.1
2026-10-13 07:44	0d6ba5f8	select	OK	8247112df86e	21.9
2026-10-13 07:44	c89443de	fit	OK	8247112df86e	1.7
2026-10-13 07:44	cfd79dbd	fit	OK	8247112df86e	1.3
2026-10-13 07:44	9fbec12f	fit	OK	8247112df86e	0.4
2026-10-13 07:44	86ce2bb7	fit	OK	8247112df86e	1.0
2026-10-13 07:44	37f9fba6	zig-perf	OK	8247112df86e	73.4
2026-10-13 07:47	00b96413	select	OK	a949f2272eef	19.8
2026-10-13 07:47	8e3fe50f	fit	OK	a949f2272eef	1.1
2026-10-13 07:47	96d4589e	fit	OK	a949f2272eef	0.9
2026-10-13 07:48	45cf6df9	competition-check	OK	ca638ea84501	42.0
2026-10-13 07:49	20bb8ce1	zig-perf	OK	ca638ea84501	11.1
2026-10-13 07:51	52e23c0b	competition-check	OK	51baedf8e4dd	124.3
2026-10-13 07:52	bc3a449c	fit	OK	51baedf8e4dd	1.9
2026-10-13 07:52	1486cae4	fit	OK	51baedf8e4dd	1.4
2026-10-13 07:52	725602cf	fit	OK	51baedf8e4dd	1.1
2026-10-13 07:52	febb9dcd	fit	OK	51baedf8e4dd	1.1
2026-10-13 07:52	7362a150	fit	OK	51baedf8e4dd	0.4
2026-10-13 07:52	564c7eb7	fit	OK	51baedf8e4dd	1.0
2026-10-13 07:53	b7259a44	fit	OK	51baedf8e4dd	2.1
2026-10-13 07:53	378cc058	fit	OK	51baedf8e4dd	1.9
2026-10-13 07:53	5ed4639e	fit	OK	51baedf8e4dd	1.5
2026-10-13 07:53	94be8f8c	fit	OK	51baedf8e4dd	1.5
2026-10-13 07:53	3fb49c58	fit	OK	51baedf8e4dd	0.4
2026-10-13 07:53	5bafceb1	fit	OK	51baedf8e4dd	1.1
2026-10-13 08:00	d4e6375c	fit	OK	123a7df1d49a	0.8
2026-10-13 08:00	5ce1f5e1	fit	OK	123a7df1d49a	1.0
2026-10-13 08:00	62445746	fit	OK	123a7df1d49a	1.3